MODULE 4 | SESSION 13

# Zero Trust and Permission Boundaries

Deny by default, approve by policy, and audit the rest.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

# The agent is useful, not trusted.

- It may need meaningful access to do useful work.
- That does not make it a trusted actor.
- Local execution is not an argument for implicit trust.
- Zero Trust is the right mental model for agentic workflows.

Permissioning is not friction. It is the control plane.

### The dangerous habit to kill

`--dangerously-skip-permissions` is not a shortcut. It is a direct bypass of the control boundary.

### What this session reframes

The modern question is not “can the agent do it?” The modern question is “why is the agent allowed to do it at all?”

# Permissioning is the control plane for agentic execution.

### Tool whitelists

Allow only the minimum tool surface needed for the task. If a tool is not justified, it should not be available.

### Read and write boundaries

Read-only analysis is a safer default. Write-capable actions should require stronger review and narrower scope.

### Approval modes

Approval is not ceremony. It is the explicit checkpoint before mutation, side effects, or downstream impact.

### AI identity scope

The agent should have an identity, ownership boundary, and role definition just like any other privileged actor.

# Least privilege should apply to AI identities the same way it applies to humans and services.

- Give the agent only the access needed for the task.
- Scope credentials to the narrowest useful boundary.
- Separate read-only from write-capable roles.
- Make approval an intentional event, not a habit of clicking through prompts.

```
Control layers to name explicitly

- identity
- permissions
- approvals
- tools
- filesystem
- connectors
- audit logs
```

# Read-only is the safe starting point. Broad trust is not.

- Read-only analysis can happen with narrower risk.
- Write access should require explicit justification.
- Destructive actions should be rare and deliberate.
- The safest default is deny first, then allow only when justified.

**Expected blocked execution**

The blocked action is not a product failure. The blocked action is proof that policy is actually doing its job.

**Common mistake**

Making the agent broadly privileged because setup is easier is operational debt disguised as convenience.

# The blocked-execution demo should prove that policy is real.

**Step 1**

Put the session in a narrow permission mode with a strict tool boundary.

**Step 2**

Ask the agent to do something outside the allowed scope.

**Step 3**

Show the denial and explain why that denial is the intended system behavior.

> The point is not drama. The point is trust boundaries made visible.

```
Teaching message

- the system denies it
- the denial is expected
- the policy is working
- this is what mature control looks like
```

# A team should be able to answer these permission questions before the agent starts work.

### Identity

Whose authority is the agent operating under, and what role is it supposed to fulfill?

### Scope

What is the narrowest useful permission set for this task?

### Approval

Which actions require explicit approval every time?

### Deny list

Where should the answer simply be “no” instead of “ask later” or “maybe if careful”?

# If the system cannot explain why the agent is allowed to do something, the agent should not be doing it.

- A useful agent is still a controlled agent.
- Zero Trust keeps that distinction real.
- Approval, whitelists, and narrow scopes are engineering controls, not admin trivia.

**Bridge to Session 14**

Session 13 controls what the agent may do. Session 14 controls what external systems the agent may reach and how those systems must defend themselves.